

100 道 MyBatis 高频面试题及答案

一、基础概念与核心架构

1. 什么是 MyBatis ? 它的核心特点是什么 ?

- 答 : MyBatis 是一款半自动化 ORM 框架 , 核心特点 :
- **灵活** : SQL 与代码分离 , 支持自定义 SQL。
- **轻量** : 无额外依赖 , 专注 SQL 优化。
- **封装 JDBC** : 简化数据库操作 , 支持动态 SQL 和结果映射。

2. MyBatis 的核心组件有哪些 ?

- 答 :
- **SqlSessionFactory** : 创建 SqlSession , 通过`SqlSessionFactoryBuilder`构建 (基于 XML 或 Java 配置)。
- **SqlSession** : 执行 SQL 的会话 , 提供`select`/`insert`/`update`/`delete`方法。
- **Executor** : 执行器 , 处理 SQL 的执行和结果集映射。
- **Mapper 接口** : 通过动态代理生成实现类 , 映射 SQL 语句。

3. MyBatis 与 Hibernate 的区别 ?

- 答：
- **自动化程度**：Hibernate 全自动化(ORM 映射复杂)，MyBatis 半自动化(手动编写 SQL)。
- **灵活性**：MyBatis 适合复杂 SQL 和性能优化，Hibernate 适合简单 CRUD 和对象建模。
- **学习成本**：Hibernate 上手难(映射配置复杂)，MyBatis 易掌握(聚焦 SQL)。

4. MyBatis 的配置文件包含哪些核心标签？

- 答：
- `<configuration>`：全局配置（如环境、插件）。
- `<environments>`：多环境配置（default 指定默认环境）。
- `<mappers>`：注册映射器（指定 XML 或接口路径）。
- `<settings>`：设置参数（如缓存、日志、延迟加载）。

5. #{}和\${}的区别？如何防止 SQL 注入？

- 答：
- `#{}` ：预编译占位符（如 `SELECT * FROM user WHERE id=#{id}`），防 SQL 注入。

- `${}`：字符串拼接（如 `SELECT * FROM ${tableName}`），存在 SQL 注入风险。
- 防注入：优先使用 `#{}` ，`${}` 仅用于非用户可控的参数（如表名、列名）。

二、映射文件与动态 SQL

6. MyBatis 的 ResultMap 有什么作用？

- 答：解决对象属性与数据库列名不一致的映射问题，通过 `<resultMap>` 定义字段映射关系，支持嵌套查询和联合结果映射。

7. 动态 SQL 标签有哪些？适用场景？

- 答：
- `<if>`：条件判断（如 `WHERE id IS NOT NULL`）。
- `<where>`：自动处理 AND/OR 前缀（避免多余的 WHERE 关键字）。
- `<foreach>`：集合遍历（批量插入/删除，如 `IN (#{item.id})`）。
- `<choose>`：分支选择（类似 switch-case）。
- `<trim>`：去除前后缀（如去除多余的逗号）。

8. 如何实现模糊查询？

- 答：
- 方式 1：`LIKE "%#{value}%"`（推荐，预编译）。
- 方式 2：`LIKE CONCAT('%', #{value}, '%')`（数据库函数，兼容性好）。
- 避免：`LIKE '%\${value}%'`（字符串拼接，存在 SQL 注入风险）。

9. MyBatis 如何处理主键自增？

- 答：
- 使用`<selectKey>`标签（适用于 MySQL）：

```

```xml
<insert id="insertUser">
 <selectKey keyProperty="id" resultType="long" order="AFTER">
 SELECT LAST_INSERT_ID()
 </selectKey>
 INSERT INTO user(name) VALUES(#{name})
</insert>
```

```

- 或通过 JDBC 原生方法`getGeneratedKeys`。

10. MyBatis 的 parameterType 和 resultType 的区别？

- 答：
- `parameterType`：输入参数类型（如 Java 对象、Map、基本类型）。

- ``resultType`` : 输出结果类型 (直接映射到 Java 对象或基本类型)。
- 若结果是复杂对象 , 需用 ``resultMap`` 定义映射关系。

三、参数处理与结果映射

11. MyBatis 如何传递多个参数 ?

- 答 :
- **方式1** : 使用 ``@Param`` 注解 (如 ``selectUser( @Param("id") Long id, @Param("name") String name)``)。
- **方式2** : 传递 Map (``new HashMap<>().put("id", 1).put("name", "Tom")``)。
- **方式3** : 传递 POJO (参数对象包含所有字段)。

12. MyBatis 的一级缓存和二级缓存的区别 ?

- 答 :
- **一级缓存** : 默认开启 , 基于 `SqlSession` (同一会话内查询相同 SQL 会缓存结果)。
- **二级缓存** : 跨 `SqlSession` , 需手动配置 (如 `<cache/>` 标签 , 支持 Ehcache、Redis 等)。

- 区别：一级缓存是会话级，二级缓存是全局级(需配置`cacheEnabled=true`)。

13. 如何配置二级缓存？

- 答：
 1. 在`mybatis-config.xml`开启二级缓存：


```

          ``xml
          <settings>
            <setting name="cacheEnabled" value="true"/>
          </settings>
          ...
          
```
 2. 在映射文件添加`<cache/>`标签。
 3. 实体类实现`Serializable`接口（可选，用于缓存存储）。

14. MyBatis 的延迟加载（懒加载）如何实现？

- 答：
- 配置`lazyLoadingEnabled=true`和`aggressiveLazyLoading=false`（默认）。
- 使用`<association>`或`<collection>`标签的`fetchType="lazy"`属性：

```

  ``xml
  <collection property="orders" column="user_id" select="getOrders"
  fetchType="lazy"/>
  ...
  
```

15. MyBatis 如何处理字段名与属性名不一致？

- 答：
- 方式 1：使用`AS`别名（SQL 中`SELECT id AS userId`）。
- 方式 2：在`resultMap`中配置映射关系：

```
``xml
<result column="user_name" property="userName"/>
``
```

- 方式 3：全局配置`mapUnderscoreToCamelCase=true`（自动转换下划线命名为驼峰）。

四、接口与动态代理

16. Mapper 接口为什么不需要实现类？

- 答：MyBatis 通过 JDK 动态代理（或 CGLIB）生成 Mapper 接口的实现类，根据接口方法名匹配映射文件中的 SQL 语句 ID。

17. @Mapper 和 @MapperScan 的区别？

- 答：
- `@Mapper`：标记单个 Mapper 接口，告诉 Spring 注入其实现类。

- `@MapperScan` :批量扫描 Mapper 接口所在包 ,替代逐个添加 `@Mapper` (推荐在 Spring Boot 中使用)。

18. MyBatis 的动态代理原理是什么？

- 答：
 1. 通过 `MapperProxyFactory` 创建 `MapperProxy` 代理对象。
 2. 代理对象拦截方法调用，根据方法名匹配映射文件中的 SQL 语句（如 `selectOne` 对应 `select` 标签的 `id`）。
 3. 调用 `SqlSession` 执行 SQL，返回结果。

19. 如何自定义 Mapper 接口的返回类型？

- 答：
- 使用 `resultType` 或 `resultMap` 指定返回类型，支持泛型（如 `List<User>`），复杂对象需定义 `resultMap` 映射字段。

20. MyBatis 如何处理存储过程？

- 答：
- 使用 `<select>` 或 `<procedure>` 标签，通过 `mode="CALL"` 调用存储过程，

配置输入输出参数：

```
``xml
<select id="callProc" statementType="CALLABLE">
    {call proc_name({inParam, mode=IN, jdbcType=VARCHAR},
{outParam, mode=OUT, jdbcType=INTEGER}}}
</select>
``
```

五、配置与整合 Spring

21. MyBatis 与 Spring 整合的核心配置类有哪些？

- 答：
- `SqlSessionFactoryBean`：创建 SqlSessionFactory，配置数据源和映射文件路径。
- `MapperScannerConfigurer`：扫描 Mapper 接口，注册为 Spring Bean（或使用`@MapperScan`注解）。

22. Spring Boot 中如何配置 MyBatis？

- 答：
 1. 添加依赖`mybatis-spring-boot-starter`。
 2. 配置`application.yml`：

```
``yaml
mybatis:
  mapper-locations: classpath:mapper/*.xml
  type-aliases-package: com.example.entity
...

```

3. 使用`@MapperScan`扫描 Mapper 接口包。

23. MyBatis 的 SqlSessionTemplate 和 SqlSession 的区别？

- 答：
- `SqlSessionTemplate`：Spring 整合 MyBatis 的核心类，线程安全，封装了 `SqlSession` 的创建和关闭（自动管理事务）。
- `SqlSession`：MyBatis 原生会话，需手动关闭（非线程安全，不建议直接使用）。

24. MyBatis 的 TypeHandler 有什么作用？

- 答：自定义类型转换（如 Java 的 Date 转数据库的 TIMESTAMP，或枚举类型映射），通过实现 `TypeHandler` 接口并注册到 MyBatis 配置中。

25. 如何处理枚举类型的映射？

- 答：
- 方式 1：使用 MyBatis 内置的 `EnumTypeHandler`（存储枚举的名称或

ordinal)。

- **方式 2** : 自定义 TypeHandler , 实现`setParameter`和`getResult`方法 , 指定枚举值与数据库字段的映射关系。

六、插件与性能优化

26. MyBatis 的插件 (Interceptor) 能拦截哪些方法 ?

- 答 : 可拦截四大对象的方法 :
- `Executor` (拦截执行方法)
- `StatementHandler` (拦截 SQL 语法构建)
- `ParameterHandler` (拦截参数处理)
- `ResultSetHandler` (拦截结果集处理)

27. 如何实现分页插件 (如 PageHelper) ?

- 答 :
 1. 注册分页插件到 MyBatis 配置 :

```
``xml
<plugins>
    <plugin interceptor="com.github.pagehelper.PageInterceptor"/>
</plugins>
```

...

2. 使用`PageHelper.startPage(pageNum, pageSize)`开启分页，结果通过`PageInfo`获取。

28. MyBatis 的批量插入如何优化？

- 答：
- 使用`<foreach>`标签批量插入（如`INSERT INTO user VALUES <foreach collection="list" item="item" separator=", "> ( #{ item. id} , #{item.name})</foreach>`）。
- 配置`jdbc`批处理（`&rewriteBatchedStatements=true`），减少网络传输次数。

29. MyBatis 的缓存失效场景有哪些？

- 答：
- 一级缓存：SqlSession 关闭或执行`insert`/`update`/`delete`（清空缓存）。
- 二级缓存：需手动配置`flushInterval`（刷新间隔）或调用`clearCache()`方法。

30. 如何优化 MyBatis 的性能？

- 答：

- **SQL 优化**：使用`EXPLAIN`分析执行计划，避免全表扫描。
- **缓存**：合理使用二级缓存（如 Redis），减少数据库压力。
- **连接池**：配置 HikariCP 或 Druid，优化数据源参数（如`maxPoolSize`）。
- **延迟加载**：按需加载关联数据，减少内存占用。

七、原理与源码分析

31. MyBatis 的初始化流程是怎样的？

- 答：
 1. 解析`mybatis-config.xml`和映射文件，生成`Configuration`对象。
 2. 创建`SqlSessionFactory`（默认`DefaultSqlSessionFactory`）。
 3. 通过`SqlSessionFactory.openSession()`获取`SqlSession`，内部创建`Executor`（默认`SimpleExecutor`）。

32. MyBatis 如何处理#{ }的参数替换？

- 答：
 - 使用`PreparedStatement`预编译，将#{ }替换为`?`占位符，通过`ParameterHandler`设置参数值，避免 SQL 注入。

33. MyBatis 的动态 SQL 是如何解析的？

- 答：
- 通过`DynamicSqlSource`解析动态 SQL 标签 (如`<if>`、`<foreach>`), 生成最终的 SQL 语句，交`Executor`执行。

34. MyBatis 的二级缓存如何实现跨会话共享？

- 答：
- 二级缓存通过`Cache`接口实现 (默认`PerpetualCache`), 存储在`Mapper`的 namespace`下，不同 SqlSession 共享同一缓存实例。

35. MyBatis 的插件开发步骤是什么？

- 答：
 1. 实现`Interceptor`接口 , 重写`intercept`、`plugin`、`setProperties`方法。
 2. 使用`@Intercepts`和`@Signature`注解指定拦截的对象和方法。
 3. 在 MyBatis 配置中注册插件。

八、常见问题与最佳实践

36. MyBatis 为什么不建议使用*作为查询字段？

- 答：
- 无法利用`resultMap`进行字段映射。
- 可能返回无用字段，增加网络传输和内存消耗。
- 不利于 SQL 优化（如索引失效）。

37. 如何处理 MyBatis 的嵌套查询性能问题？

- 答：
- 避免多层嵌套（如`select="getUser" column="user\_id"`），改用`JOIN`一次性查询。
- 配置`fetchType="eager"`（立即加载），或控制嵌套查询的层级。

38. MyBatis 的批量删除如何实现？

- 答：
- 使用`<foreach>`标签拼接`IN`条件（如`DELETE FROM user WHERE id IN <foreach collection="ids" item="id" separator=",">#{id}</foreach>`）。

39. MyBatis 如何处理事务？

- 答：
- 依赖 Spring 的事务管理 (`@Transactional` 注解) , MyBatis 的 `SqlSession` 在事务提交时自动提交，回滚时回滚。

40. MyBatis 的日志如何配置？

- 答：
- 全局配置 `logImpl` (如 `LOG4J`、`SLF4J`) :

```

<<xml
<settings>
    <setting name="logImpl" value="LOG4J"/>
</settings>
...

```
- 或在映射文件中配置 `<select id="..." log="debug">`。

九、高级特性与扩展

41. MyBatis 的多数据源如何实现？

- 答：
- 使用动态数据源 (如 `AbstractRoutingDataSource`) , 根据条件切换数据源 (如 `@DataSource("slave")` 注解) 。

42. MyBatis 的枚举类型处理器如何注册？

- 答：
- 在`mybatis-config.xml`中添加：

```
    <<xml
    <typeHandlers>
                                                <typeHandler
handler="com.example.handler.EnumTypeHandler"/>
    </typeHandlers>
    <<
```

43. MyBatis 的插件能修改 SQL 吗？

- 答：可以，通过拦截`StatementHandler`的`prepare`方法，修改 SQL 语句（如添加分页条件、数据权限过滤）。

44. MyBatis 的二级缓存如何与 Redis 集成？

- 答：
- 实现 MyBatis 的`Cache`接口，使用 Redis 存储缓存数据，配置`cache-ref`引用自定义缓存。

45. MyBatis 的延迟加载为什么会出现 N+1 问题？

- 答：

- 主查询加载主对象，每个子对象触发一次子查询，导致 N+1 次 SQL(N 为子对象数量)。
- 解决：使用`JOIN`一次性加载，或批量延迟加载（如`BatchExecutor`）。

十、源码与深度原理（持续更新至 100 题）

46. MyBatis 的 SqlSession 有哪些实现类？区别是什么？

- 答：
- `DefaultSqlSession`：默认实现，支持自动提交(`autoCommit=false`默认)。
- `SqlSessionManager`：线程安全，用于多线程环境（不推荐，建议用 Spring 管理）。

47. MyBatis 的 Executor 有哪些类型？

- 答：
- `SimpleExecutor`：默认执行器，每次执行 SQL 创建新 Statement。
- `ReuseExecutor`：重用 Statement（提升性能）。
- `BatchExecutor`：批量执行 SQL（减少数据库交互次数）。

48. MyBatis 如何处理数据库方言？

- 答：
- 通过`DatabaseIdProvider`识别数据库类型,在映射文件中使用`databaseId`属性区分 SQL (如`<select databaseId="mysql">`)。

49. MyBatis 的@Param 注解的作用是什么？

- 答：
- 为多参数指定名称，避免参数混淆 (MyBatis 会将参数封装为`ParamMap`，键为`param1`、`param2`或自定义名称)。

50. MyBatis 的一级缓存为什么是会话级？

- 答：
- 基于`PerpetualCache`实现，存储在`SqlSession`的`localCache`中，同一会话内重复查询直接取缓存，不同会话不共享。

十一、高频陷阱与细节

51. MyBatis 的 insert 语句返回值是什么？

- 答：
- 自增主键：通过`selectKey`获取主键值。

- 非自增：返回影响的行数（`int`类型）。

52. MyBatis 的模糊查询为什么不能用#{ }？

- 答：可以用，如`LIKE concat('%', #{value}, '%')`，避免直接拼接字符串（`LIKE '%\${value}%'`有注入风险）。

53. MyBatis 的@InsertProvider 如何使用？

- 答：
- 动态生成SQL（如`@InsertProvider(type = SqlProvider.class, method = "insertSql")`），返回 SQL 字符串。

54. MyBatis 的枚举字段如何存储到数据库？

- 答：
- 存储枚举的`name()`或`ordinal()`，或自定义 TypeHandler 存储指定值（如枚举的自定义属性）。

55. MyBatis 的多数据库支持如何实现？

- 答：
- 配置多个数据源，通过动态切换数据源注解（如`@DS("master")`）或 AOP

实现。

十二、性能与调优实战

56. MyBatis 的批量操作如何避免 OOM ?

- 答：
- 分批处理（如每次处理 1000 条），避免一次性加载大量数据到内存。

57. MyBatis 的缓存穿透如何处理？

- 答：
- 缓存空结果（设置短过期时间），或使用布隆过滤器过滤无效请求。

58. MyBatis 的延迟加载超时如何配置？

- 答：
- 全局配置`lazyLoadTriggerTime`（默认 30000ms）：

```
```xml
<setting name="lazyLoadTriggerTime" value="5000"/>
```
```

59. MyBatis 的二级缓存如何设置过期时间？

- 答：
- 使用第三方缓存 (如 Ehcache) , 在配置文件中设置`timeToLiveSeconds`属性。

60. MyBatis 的 SQL 注入漏洞如何彻底避免？

- 答：
- 永远使用#{ }代替\${ } (除表名、列名等必须拼接的场景) , 对用户输入做过滤。

十三、Spring 整合与源码

61. MyBatis 的@Mapper 接口如何被 Spring 识别？

- 答：
- `MapperScannerConfigurer` 扫描包路径 , 为每个接口生成`MapperFactoryBean` , 注册为 Spring Bean。

62. MyBatis 的 SqlSession 如何与 Spring 事务结合？

- 答：
- Spring 的`TransactionSynchronizationManager`管理 SqlSession 的生命周期 , 事务提交时关闭会话。

63. MyBatis 的@One 和@Many 注解的区别？

- 答：
- `@One`：一对一关联(如 `@One(select = "getUser", column = "user_id")`)。
- `@Many`：一对多关联(如 `@Many(select = "getOrders", column = "id")`)。

64. MyBatis 的批量操作如何返回主键？

- 答：
- 不支持批量返回自增主键(需逐条插入)，或使用数据库函数(如 MySQL 的 `LAST_INSERT_ID()` 在批量插入时返回第一个主键)。

65. MyBatis 的 TypeAlias 有什么作用？

- 答：
- 为实体类设置别名(如 `<typeAlias alias="user" type="com.example.User"/>`)，简化映射文件配置。

十四、高频考点速答 (81-100)

81. MyBatis 的@UpdateProvider 如何动态拼接 SQL？

- 答：通过`@UpdateProvider`指定 SQL 提供方法，返回拼接好的 SQL 字符串（需注意参数安全）。

82. MyBatis 的嵌套结果如何映射？

- 答：使用`resultMap`的`<association>`和`<collection>`标签，通过`column`和`property`指定关联字段。

83. MyBatis 的多租户如何实现？

- 答：通过插件在 SQL 中注入租户 ID（如`WHERE tenant\_id = #{currentTenantId}`），或动态修改数据源。

84. MyBatis 的枚举类型处理器如何处理数据库存储？

- 答：重写`setParameter`和`getResult`方法，实现枚举值与数据库字段的双向转换。

85. MyBatis 的`@Param("ids") List<Long> ids`如何映射？

- 答：MyBatis 自动将 List 转换为`collection="ids"`，在`<foreach>`中使用`item="id"`遍历。

86. MyBatis 的模糊查询性能差如何优化？

- 答：对搜索字段建立全文索引，或使用数据库的全文搜索功能（如 MySQL 的 FULLTEXT 索引）。

87. MyBatis 的二级缓存如何配置过期策略？

- 答：依赖第三方缓存框架（如 Redis）的过期策略，MyBatis 自身不提供复杂过期配置。

88. MyBatis 的批量插入如何处理自增主键？

- 答：批量插入不支持返回多个自增主键，需逐条插入或使用数据库特定语法（如 MySQL 的 `LAST_INSERT_ID()`）。

89. MyBatis 的 `@DeleteProvider` 如何处理多个参数？

- 答：通过 `@Param` 注解命名参数，在 SQL 提供方法中拼接条件（如 `delete from user where id in (#{ids})`）。

90. MyBatis 的延迟加载在什么场景下失效？

- 答：当调用对象的序列化方法、哈希方法（如 `hashCode()`）或直接访问关联对象时，触发延迟加载。

91. MyBatis 的@ResultMap 如何引用其他映射？

- 答：使用`extends`属性继承其他ResultMap（如`<resultMap id="userMap" extends="baseMap">`）。

92. MyBatis 的插件如何获取当前执行的 SQL？

- 答：通过拦截`StatementHandler`的`prepare`方法，获取`boundSql.getSql()`获取最终 SQL。

93. MyBatis 的@Insert 如何返回生成的主键？

- 答：使用`<selectKey>`标签（`order="AFTER"`），或通过`KeyGenerator`接口实现。

94. MyBatis 的 TypeHandler 如何处理 NULL 值？

- 答：重写`getResult`方法时，判断结果是否为 NULL，返回对应的 Java 类型默认值。

95. MyBatis 的二级缓存如何处理并发问题？

- 答：依赖缓存框架的并发控制（如 Ehcache 的读写锁），MyBatis 自身不处

理。

96. MyBatis 的@Param 注解不使用会怎样？

- 答：多参数会被封装为`ParamMap`，键为`param1`、`param2`，使用时需按顺序或通过索引访问。

97. MyBatis 的批量操作如何提高数据库吞吐量？

- 答：使用`BatchExecutor`（`ExecutorType.BATCH`），并配置`rewriteBatchedStatements=true`。

98. MyBatis 的@One 注解的 fetchType 有什么值？

- 答：`eager`（立即加载）和`lazy`（延迟加载，默认），根据业务场景选择。

99. MyBatis 的映射文件如何引用其他文件？

- 答：使用`<include refid="otherMapper.selectById"/>`引用其他映射文件中的 SQL 片段。

100. MyBatis 的最佳实践有哪些？

- 答：

- 避免使用*，明确查询字段。
- 合理使用缓存（一级缓存默认开启，二级缓存按需配置）。
- 复杂 SQL 写在映射文件，简单 CRUD 用接口方法（结合@Select 等注解）。
- 配置`mapUnderscoreToCamelCase=true`，简化字段映射。
- 利用插件实现分页、数据权限、SQL 日志等通用功能。

总结

MyBatis 面试需重点掌握以下核心：

1. **基础配置**：核心组件、映射文件结构、动态 SQL 标签的使用场景。
2. **参数与结果**：#{ }与\${ }的区别、ResultMap 的映射规则、多参数传递方式。
3. **高级特性**：缓存机制（一级/二级缓存）、延迟加载、插件开发、与 Spring 的整合。
4. **原理与优化**：动态代理原理、SQL 执行流程、性能优化手段（批量操作、分页、索引优化）。
5. **实战经验**：处理字段映射不一致、防止 SQL 注入、解决 N+1 问题、多数据源配置。

建议结合实际项目中的 MyBatis 使用场景，深入理解源码（如`Configuration`、`Executor`、`MapperProxy`），面试时通过具体案例说明问题解决思路，例如：

- 如何设计一个支持多数据库方言的 MyBatis 插件？
- 生产环境中遇到 MyBatis 性能瓶颈时，你的排查和优化步骤是什么？

通过模拟面试和代码调试，加深对 MyBatis 底层机制的理解，提升问题分析和解决问题的能力。